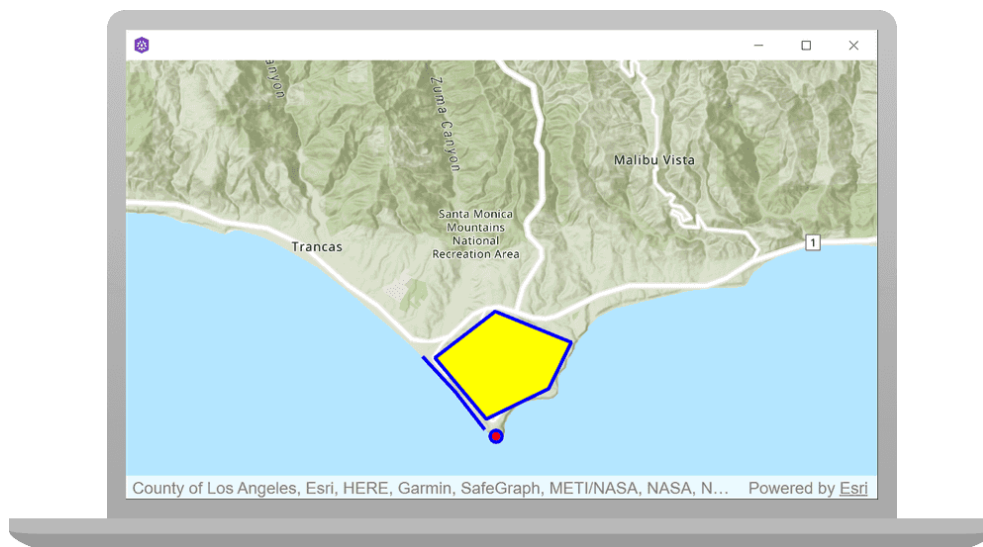


Add a point, line, and polygon

Learn how to display point, line, and polygon graphics in a map.



You typically use graphics to display geographic data that is not connected to a database and that is not persisted, like highlighting a route between two locations, displaying a search buffer around a selected point, or showing the location of map coordinates entered by the user. Graphics are composed of a geometry, symbol, and attributes.

In this tutorial, you display points, lines, and polygons on a map as graphics.

Prerequisites

Before starting this tutorial:

- 1 You need an [ArcGIS Location Platform](#) or [ArcGIS Online](#) account.

- 2 | Your system meets the [system requirements](#).
- 3 | The [ArcGIS Maps SDK for Qt](#), version 200.8.0 or later is installed.
- 4 | The [Qt 6.8.2 software development framework](#) is installed.

i Note: For simplicity, the steps for the ArcGIS Maps SDK for Qt tutorials are shown for creating Windows Desktop apps on a Windows desktop computer. Qt being cross-platform, allows for the flexibility to create apps on multiple configurations, such as Android, Linux, and macOS. If you are developing a non-Windows machine, you will need to make adjustments as you work through the tutorials.

When completing the tutorials, it is recommended that following **Qt Creator** setting is used:

- In the menu bar, choose **Edit > Preferences...**
- In the **Preferences- Qt Creator** dialog, on the left choose **C++**
- Make the **Coding Style** tab active
- In the **ClangFormat settings** section, ensure the **Formatting Mode** is set to **Use built-in indenter**

Develop or Download

You have two options for completing this tutorial:

- 1 | [Option 1: Develop the code](#) or
- 2 | [Option 2: Download the completed solution](#)

Option 1: Develop the code

To start the tutorial, complete the [Display a map](#) tutorial. This creates a map to display the Santa Monica Mountains in California using the topographic basemap from the [ArcGIS Basemap Styles service](#).

Open a Qt Creator project

- 1 | Open the project you created by completing the [Display a map](#) tutorial.
- 2 | Continue with the following instructions to display a point, line, and polygon in the map.

Add GraphicsOverlay class, declare member function

[GraphicsOverlay](#) is a container for temporary graphics to display on your map view. The graphics drawn in graphics overlays are created at runtime and are not persisted when your application closes. Learn more about [GraphicsOverlay](#).

- 1 | In the **display_a_map** project, double click **Headers > Display_a_map.h** to open the file. Add the [GraphicsOverlay](#) class to the `namespace ArcGISRuntime` declaration.

Display_a_map.h

```
class Map;  
class MapQuickView;  
  
class GraphicsOverlay;  
}
```

- 2 Add the `createGraphics` private member function declaration. Then save and close the header file.

Display_a_map.h

Collapse



```
// Copyright 2022 ESRI
//
// ALL rights reserved under the copyright laws of
// and applicable international laws, treaties, and
//
// You may freely redistribute and use this sample
// without modification, provided you include the c
// notice and use restrictions.
//
// See the Sample code usage restrictions document
//

#ifndef DISPLAY_A_MAP_H
#define DISPLAY_A_MAP_H

namespace Esri::ArcGISRuntime {

class Map;
class MapQuickView;

class GraphicsOverlay;
}

#include <QObject>

Q_MOC_INCLUDE("MapQuickView.h")

class Display_a_map : public QObject
{
    Q_OBJECT

    Q_PROPERTY(Esri::ArcGISRuntime::MapQuickView* n

public:
    explicit Display_a_map(QObject* parent = nullptr)
    ~Display_a_map() override;

signals:
    void mapViewChanged();

private:
    Esri::ArcGISRuntime::MapQuickView* mapView() cc
```

```

void setMapView(Esri::ArcGISRuntime::MapQuickVi
void setupViewpoint();

void createGraphics(Esri::ArcGISRuntime::Graphi

Esri::ArcGISRuntime::Map* m_map = nullptr;
Esri::ArcGISRuntime::MapQuickView* m_mapView =
};

#endif // DISPLAY_A_MAP_H

```

Collapse ^

Create a graphics overlay

A graphics overlay is a container for graphics. It is added to a map view to display graphics on a map. You can add more than one graphics overlay to a map view. Graphics overlays are displayed on top of all the other layers.

- 1 Double click on **Sources > Display_a_map.cpp** to open the file. Include the classes shown.

Display_a_map.cpp

```

#include "Display_a_map.h"
#include "Map.h"
#include "MapQuickView.h"
#include "MapTypes.h"
#include "Point.h"
#include "SpatialReference.h"
#include <QFuture>
#include "Viewpoint.h"

#include "Graphic.h"
#include "GraphicListModel.h"
#include "GraphicsOverlay.h"
#include "GraphicsOverlayListModel.h"
#include "PolylineBuilder.h"
#include "PolygonBuilder.h"
#include "SimpleFillSymbol.h"
#include "SimpleLineSymbol.h"
#include "SimpleMarkerSymbol.h"
#include "SymbolTypes.h"

```

- 2 In the `Display_a_map::setMapView` member function, add three lines of code to create a `GraphicsOverlay`, call the `createGraphics` method (implemented in following steps), and append the overlay to the map view.

Display_a_map.cpp

```
// Set the view (created in QML)
void Display_a_map::setMapView(MapQuickView* mapView)
{
    if (!mapView || mapView == m_mapView)
    {
        return;
    }

    m_mapView = mapView;
    m_mapView->setMap(m_map);
    setupViewpoint();

    GraphicsOverlay* overlay = new GraphicsOverlay(
        createGraphics(overlay));
    m_mapView->graphicsOverlays()->append(overlay);
}
```

- 3 Create a new method named `Display_a_map::createGraphics`, right after the `Display_a_map::setupViewpoint` method.

Display_a_map.cpp

```
void Display_a_map::setupViewpoint()
{
    const Point center(-118.80543, 34.02700, SpatialReference::WGS84);
    const Viewpoint viewpoint(center, 100000.0);
    m_mapView->setViewpointAsync(viewpoint);
}

void Display_a_map::createGraphics(GraphicsOverlay* overlay)
{
    // ... (implementation details) ...
}

}
```

Add a point graphic

A [point graphic](#) is created using a [point](#) and a [marker symbol](#). A point is defined with x and y coordinates for longitude and latitude coordinates, and a [spatial reference](#). The spatial reference is WGS84.

- 1 Create a [Point](#) and a [SimpleMarkerSymbol](#). To create the [Point](#), provide longitude (x) and latitude (y) coordinates, and a [SpatialReference](#).

Point graphics support a number of symbol types such as [SimpleMarkerSymbol](#), [PictureMarkerSymbol](#), [TextSymbol](#), and [TextMarkerSymbol](#). See [Symbol](#) in the API documentation to learn more about symbols.

Display_a_map.cpp

```
void Display_a_map::setupViewpoint()
{
    const Point center(-118.80543, 34.02700, SpatialReference::WGS84);
    const Viewpoint viewpoint(center, 100000.0);
    m_mapView->setViewpointAsync(viewpoint);
}

void Display_a_map::createGraphics(GraphicsOverlay* overlay)
{
    // Create a point
    const Point dume_beach(-118.80657463861, 34.00000, SpatialReference::WGS84);
    // Create symbols for the point
    SimpleLineSymbol* point_outline = new SimpleLineSymbol(SpatialReference::WGS84, Color::red, 2);
    SimpleMarkerSymbol* point_symbol = new SimpleMarkerSymbol(SpatialReference::WGS84, Color::red, SimpleMarkerSymbol::SHAPE_CIRCLE, 10);
    point_symbol->setOutline(point_outline);

    // Create a graphic to display the point with its symbol
    Graphic* point_graphic = new Graphic(dume_beach, point_symbol);
    // Add point graphic to the graphics overlay
    overlay->graphics()->append(point_graphic);
}
```

2 | Press **Ctrl + R** to run the app.

You should see a point graphic at Point Dume State Beach, California.

Add a polyline graphic

A line graphic is created using a [polyline](#) and a [line symbol](#). A polyline is defined as a sequence of points.

Polylines have one or more distinct parts. Each part is defined by two points. To create a continuous line with just one part, use the [Polyline](#) constructor. To create a polyline with more than one part, use a [PolylineBuilder](#). Polyline graphics support a number of symbol types, such as [SimpleLineSymbol](#) and [TextSymbol](#). See [Symbol](#) in the API documentation to learn more about symbols.

1 | Create a [Polyline](#) and a [SimpleLineSymbol](#).

To create the [Polyline](#), create a new [PointCollection](#) with a [SpatialReference](#) and use [PolylineBuilder](#) to add a new [Point](#) objects to it. Add the highlighted code.

Display_a_map.cpp

```
// Create a graphic to display the point with i
Graphic* point_graphic = new Graphic(dume_beach
// Add point graphic to the graphics overlay
overlay->graphics()->append(point_graphic);

// Create a Line
PolylineBuilder* polyline_builder = new Polylin
polyline_builder->addPoint(-118.8215, 34.0140);
polyline_builder->addPoint(-118.8149, 34.0081);
polyline_builder->addPoint(-118.8089, 34.0017);
// Create a symbol for the line
SimpleLineSymbol* line_symbol = new SimpleLineS

// Create a graphic to display the line with it
Graphic* polyline_graphic = new Graphic(polylin
// Add Line graphic to the graphics overlay
```



```
overlay->graphics()->append(polyline_graphic);
```

- 2 Press **Ctrl + R** to run the app.

You should see a point and a line graphic along Westward Beach.

Add a polygon graphic

A polygon graphic is created using a [polygon](#) and a [fill symbol](#). A polygon is defined as a sequence of points that describe a closed boundary.

Polygons have one or more distinct parts. Each part is a sequence of points describing a closed boundary. For a single area with no holes, you can use [Polygon](#) to create a polygon with just one part. To create a polygon with more than one part, use [PolygonBuilder](#).

Polygon graphics support a number of symbol types such as [SimpleFillSymbol](#), [PictureFillSymbol](#), [SimpleMarkerSymbol](#), and [TextSymbol](#). See [Symbol](#) in the API documentation to learn more about symbols.

- 1 Create a [Polygon](#) and a [SimpleFillSymbol](#). To create the [Polygon](#), create a new [PointCollection](#) with a [SpatialReference](#) and use [PolygonBuilder](#) to add new [Point](#) objects to it. Add the highlighted code.

Display_a_map.cpp

Collapse



```
// Copyright 2022 ESRI
//
// ALL rights reserved under the copyright laws of
// and applicable international laws, treaties, and
//
// You may freely redistribute and use this sample
// without modification, provided you include the c
// notice and use restrictions.
//
// See the Sample code usage restrictions document
//
```

```

#include "Display_a_map.h"
#include "Map.h"
#include "MapQuickView.h"
#include "MapTypes.h"
#include "Point.h"
#include "SpatialReference.h"
#include <QFuture>
#include "Viewpoint.h"

#include "Graphic.h"
#include "GraphicListModel.h"
#include "GraphicsOverlay.h"
#include "GraphicsOverlayListModel.h"
#include "PolylineBuilder.h"
#include "PolygonBuilder.h"
#include "SimpleFillSymbol.h"
#include "SimpleLineSymbol.h"
#include "SimpleMarkerSymbol.h"
#include "SymbolTypes.h"

#include <QUrl>

using namespace Esri::ArcGISRuntime;
Display_a_map::Display_a_map(QObject* parent /* = r
    QObject(parent),
    m_map(new Map(BasemapStyle::ArcGISTopographic,
{
}

Display_a_map::~Display_a_map()
{
}

MapQuickView* Display_a_map::mapView() const
{
    return m_mapView;
}

void Display_a_map::setupViewpoint()
{
    const Point center(-118.80543, 34.02700, SpatialReference::WGS84);
    const Viewpoint viewpoint(center, 100000.0);
    m_mapView->setViewpointAsync(viewpoint);
}

void Display_a_map::createGraphics(GraphicsOverlay

```

```

{

    // Create a point
    const Point dume_beach(-118.80657463861, 34.006
    // Create symbols for the point
    SimpleLineSymbol* point_outline = new SimpleLir
    SimpleMarkerSymbol* point_symbol = new SimpleMa
    point_symbol->setOutline(point_outline);

    // Create a graphic to display the point with i
    Graphic* point_graphic = new Graphic(dume_beach
    // Add point graphic to the graphics overlay
    overlay->graphics()->append(point_graphic);

    // Create a line
    PolylineBuilder* polyline_builder = new Polylin
    polyline_builder->addPoint(-118.8215, 34.0140);
    polyline_builder->addPoint(-118.8149, 34.0081);
    polyline_builder->addPoint(-118.8089, 34.0017);
    // Create a symbol for the line
    SimpleLineSymbol* line_symbol = new SimpleLineS

    // Create a graphic to display the line with it
    Graphic* polyline_graphic = new Graphic(polylin
    // Add line graphic to the graphics overlay
    overlay->graphics()->append(polyline_graphic);

    // Create a list of points to make up the polyg
    const QList<Point> points = {
        Point(-118.8190, 34.0138),
        Point(-118.8068, 34.0216),
        Point(-118.7914, 34.0164),
        Point(-118.7960, 34.0086),
        Point(-118.8086, 34.0035),
    };
    // Create a polygon using the list of points at
    PolygonBuilder* polygon_builder = new PolygonBu
    polygon_builder->addPoints(points);
    // Create symbols for the polygon
    SimpleLineSymbol* polygon_line_symbol = new Sin
    SimpleFillSymbol* fill_symbol = new SimpleFillS
    // Create a graphic to display the polygon with
    Graphic* polygon_graphic = new Graphic(polygon_
    // Add polygon graphic to the graphics overlay
    overlay->graphics()->append(polygon_graphic);

}

```

```
// Set the view (created in QML)
void Display_a_map::setMapView(MapQuickView* mapView)
{
    if (!mapView || mapView == m_mapView)
    {
        return;
    }

    m_mapView = mapView;
    m_mapView->setMap(m_map);
    setupViewpoint();

    GraphicsOverlay* overlay = new GraphicsOverlay(
        createGraphics(overlay));
    m_mapView->graphicsOverlays()->append(overlay);

    emit mapViewChanged();
}
```

2 Press **Ctrl + R** to run the app. Collapse ^

You should see a point, line, and polygon graphic around Mahou Riviera in the Santa Monica Mountains.

Alternatively, you can download the tutorial solution, as follows.

Option 2: Download the solution

- 1 Click the [Download solution](#) link under [Solution](#) and unzip the file to a location on your machine.
- 2 Open the project in Qt Creator.

Note: The **Configure Project** window may appear for you to specify the Qt **Kit** to use to compile the project. If you are using ArcGIS Maps SDK for Qt (**v200.7**) or greater, check on the Qt **Kit** called **Desktop Qt 6.8.2 MSVC2022 64bit** before

clicking the **Configure Project** button at the bottom of the page.

Since the downloaded solution does not contain authentication credentials, you must set up authentication to create the developer credentials and add them to the project.

Set up authentication

To access the secure [ArcGIS location services](#) used in this tutorial, you must implement [API key authentication](#) using an [ArcGIS Location Platform](#) or an [ArcGIS Online](#) account.

API key authentication

Create a new API key [access token](#) with [privileges](#) to access the secure resources used in this tutorial.

- 1 Complete the [Create an API key](#) tutorial and create an API key with the following [privilege\(s\)](#):
 - **Privileges**
 - **Location services > Basemaps**
- 2 Copy and paste the **API key access token** into a safe location. It will be used in a later step.

Set developer credentials in the solution

To allow your app users to access [ArcGIS location services](#), use the developer credentials that you created in the [Set up authentication](#) step to authenticate requests for resources.

API key authentication

- 1 | In the **Projects** window, go to the **Sources** folder
- 2 | Open the *main.cpp* file.
- 3 | Modify the code to set the `accessToken` using your **API key access token**.

main.cpp

```
// 2. API key authentication: Get a Long-L  
// ArcGIS Location services. Go to the tut  
// Copy the API Key access token.
```

```
const QString accessToken = QString("");
```

- 4 | Save and close the file.

Best Practice: The access token is stored directly in the code as a convenience for this tutorial. Do not store credentials directly in source code in a production environment.

Run the solution

Press **Ctrl + R** to run the app.

You should see a point, line, and polygon graphic around Mahou Riviera in the Santa Monica Mountains.

What's next?

Learn how to use additional [API features](#), [ArcGIS location services](#), and [ArcGIS tools](#) in these tutorials:

- [Add a feature layer](#)
- [Search for an address](#)

- [Find a route and directions](#)
- [Display a web map](#)
- [Display a scene](#)

PREVIOUS

< [Display a map](#)

NEXT

[Display a web map](#) >

Copyright © 2025 Esri. All rights reserved.

[Privacy](#) • [Manage cookies](#) • [Terms of Use](#)

[Sign up to receive the latest news from Esri](#)



esri[®]

**THE
SCIENCE
OF
WHERE[®]**